

AI 데이터 분석 · 6일차

리스트 — 여러 값을 한 목록에

담고 · 꺼내고 · 자르고 · 넣고 빼고 · 정렬하기

[] · 인덱싱 · 슬라이싱 · **append/insert/extend** · **remove/pop/del** · **sort**

교안 04_01(리스트) · Python 09_list.py · Practice 04_01 정리

각 메서드마다 “응용” — 실전에서 어떻게 쓰이는지까지

2026-07-28 AI 데이터 분석 교육 정리 (6일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

지금까지는 값 하나에 이름 하나(변수)였습니다. 오늘부터는 값 여러 개를 하나의 목록으로 묶는 **리스트**를 다룹니다. 문자열에서 배운 인덱싱·슬라이싱이 그대로 통하고, 거기에 “값을 넣고 빼는” 기능이 더해집니다.

이번 회차부터 각 항목에 **응용** 섹션이 추가되었습니다. 문법 하나를 배우면 “그래서 실전에서 어떻게 쓰는지” 까지 바로 이어서 익힙니다.

자료형 list **메서드** append·sort **내장함수** len·sum **연산자** in **키워드** del **개념** 참조·복사

연결

문자열에서 익힌 `s[0]·s[1:4]·in·count/index`가 리스트에서도 거의 똑같이 동작합니다. 다른 점은 **리스트는 값을 바꿀 수 있다(가변)**는 것 — 그래서 `append·remove` 같은 “수정” 메서드가 생깁니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	왜 리스트인가 — 변수의 한계	변수 100개 문제 · 서랍장 비유 · list 자료형
STEP 1	리스트 만들기과 크기	[] · 혼합 자료형 · 빈 리스트 · len() · type()
STEP 2	인덱싱 — 값 하나 꺼내기	0부터 · 음수 인덱스(-1=최신) · IndexError
STEP 3	슬라이싱 — 구간 잘라내기	[start:end:step] · 생략 · 결과는 새 리스트
STEP 4	값 바꾸기와 존재 확인	temps[i]=값 · in / not in · index()
STEP 5	값 추가·제거	append·insert·extend / remove·pop·del
STEP 6	정렬과 탐색	sort·reverse·count·index · sorted()
STEP 7	리스트 복사와 메모리 (심화)	참조 vs copy() — 오늘의 가장 중요한 함정

CHAPTER 0

왜 리스트인가 — 변수의 한계

값 하나에 이름 하나로는 감당이 안 될 때

측정값이 100개라면 변수를 100개 만들어야 할까요? 평균을 구하려면 $t_1 + t_2 + \dots + t_{100}$ 을 다 적어야 할까요?
이 불편을 푸는 도구가 리스트입니다.

개념 변수 vs 리스트 받침대 vs 서랍장

정의 변수는 값 하나를 담은 “받침대”라 새 값을 넣으면 이전 값이 사라집니다. 리스트는 칸이 여러 개인 “서랍장”이라 값을 옆에 계속 쌓고, 담은 순서가 그대로 유지됩니다.

```
# 변수 방식 — 값 5개면 이름도 5개
t1, t2, t3, t4, t5 = 25, 26, 24, 28, 27
print((t1+t2+t3+t4+t5)/5) # 평균 계산이 값 개수만큼 길어짐

# 리스트 방식 — 이름 하나로 전부
temps = [25, 26, 24, 28, 27]
print(sum(temps) / len(temps)) # 개수가 100개여도 코드는 그대로
```

```
▶ 26.0
▶ 26.0
```

왜 쓰나 값의 개수가 늘어나도 코드 길이가 그대로입니다. 설비 데이터는 값이 수천·수만 개라, 리스트 없이는 분석 자체가 불가능해요.

응용 `sum()`과 `len()`을 조합하면 평균이 한 줄로 나옵니다 — `sum(리스트)/len(리스트)`. 여기에 `max()`·`min()`을 더하면 기본 통계가 완성됩니다.

```
temps = [25, 26, 24, 28, 27]
print(sum(temps))          # 130 (합계)
print(max(temps))          # 28 (최댓값)
print(min(temps))          # 24 (최솟값)
print(sum(temps)/len(temps)) # 26.0 (평균)
```

```
▶ 130
▶ 28
▶ 24
▶ 26.0
```

구분	변수	리스트
담는 값	값 하나	여러 값
새 값 넣으면	이전 값 사라짐	옆에 추가됨
순서	없음	담은 순서 유지
비유	받침대	서랍장

리스트가 필요한 실제 상황

설비 데이터를 다루다 보면 “여러 값을 한 덩어리로” 다뤄야 하는 순간이 계속 나옵니다. 오늘 배우는 리스트가 그때마다 쓰입니다.

상황	리스트로 어떻게	오늘 배울 도구
1분마다 온도 수집	빈 리스트에 계속 쌓기	<code>append()</code>
최근 3개만 확인	뒤에서 세 개 잘라내기	<code>temps[-3:]</code>
이상값 999 찾아 수정	위치 찾아 그 자리만 교체	<code>in · index()</code>
가장 높은 온도 3개	큰 값부터 정렬 후 자르기	<code>sort · [:3]</code>
상태별 건수 집계	값이 몇 번 나오는지 세기	<code>count()</code>

개념

리스트는 “**값들의 컨테이너**” 입니다. 앞으로 배울 조건문·반복문, 그리고 pandas의 데이터프레임까지 전부 이 “여러 값을 한 덩어리로 다루는” 사고방식 위에 세워집니다. 오늘이 그 출발점이에요.

STEP 1

리스트 만들기과 크기

대괄호에 쉼표로 나열하면 끝

자료형 리스트 만들기 [] 값을 쉼표로 나열

정의 대괄호 [] 안에 값을 쉼표로 나열하면 리스트가 만들어집니다. 글자를 담을 땐 따옴표가 필수입니다.

```
temps = [25, 26, 24, 28, 27]      # 숫자 리스트
machines = ["펌프", "모터", "압축기"] # 글자 리스트
mixed = ["펌프", 78, True]         # 종류 섞어도 됨
empty = []                         # 빈 리스트
print(temps)
print(machines)
```

▶ [25, 26, 24, 28, 27]
▶ ['펌프', '모터', '압축기']

왜 쓰나 리스트는 담는 값의 종류를 가리지 않지만, 실무 분석에서는 같은 종류만 담는 경우가 대부분입니다(온도는 온도끼리). 섞여 있으면 평균 같은 계산이 안 되니까요.

응용 빈 리스트 []로 시작해 값을 하나씩 모으는 패턴이 실전에서 가장 많이 쓰입니다. 센서에서 값이 들어올 때마다 append로 쌓아 나가는 식이죠(STEP 5에서 이어집니다).

```
results = []          # 빈 목록으로 시작
results.append(30)     # 값이 들어올 때마다
results.append(31)
print(results)        # [30, 31]
```

▶ [30, 31]

내장함수 len() · type() 개수 세기 · 종류 확인

정의 len()은 리스트에 담긴 값의 개수를, type()은 그 값의 자료형을 알려줍니다.

```
temps = [37, 38, 20, 55, 33]
print(len(temps))          # 5
print(len([]))             # 0 (빈 리스트)
print(type(temps))         # <class 'list'>
print(type(temps[0]))      # <class 'int'> ← 요소 하나는 int!
```

```
▶ 5
▶ 0
▶ <class 'list'>
▶ <class 'int'>
```

왜 쓰나 `len()`은 문자열에서 쓰던 것과 똑같습니다. 리스트 자체는 `list`지만, 그 안의 요소 하나를 꺼내면 그 값의 자료형(`int·str·float`)이 나온다는 점이 중요합니다.

응용 `len()`은 “마지막 인덱스 = `len-1`”을 계산하거나, 데이터가 예상 개수만큼 들어왔는지 검증할 때 씁니다. 개수를 변수에 담아두면 반복해서 쓰기 좋습니다.

```
temps = [37, 38, 20, 55, 33]
n = len(temps)
print(temps[n-1])         # 33 (마지막 값 — 음수 인덱스 없이)
print(n == 5)             # True (데이터 개수 검증)
```

```
▶ 33
▶ True
```

STEP 2

인덱싱 — 값 하나 꺼내기

문자열에서 배운 그 규칙 그대로

문법 인덱싱 리스트[번호] 0부터 시작

정의 리스트 이름 뒤 대괄호에 순서 번호를 넣어 값 하나를 꺼냅니다. 첫 번째 값이 0번, 마지막은 (개수-1)번.

```
temps = [25, 26, 24, 28, 27]
#       0   1   2   3   4
print(temps[0])    # 25 (첫 번째)
print(temps[2])    # 24 (세 번째)
```

```
▶ 25
▶ 24
```

왜 쓰나 문자열의 `word[0]`과 완전히 같은 규칙입니다. 0부터 센다는 것만 확실히 잡으면 헷갈릴 일이 없어요.

자주 하는 실수

없는 번호를 쓰면 `IndexError: list index out of range`. 5개짜리 리스트의 마지막은 4번이고 5번은 없습니다 — “무심코 하나 크게” 쓰는 게 가장 흔한 원인이예요.

문법 음수 인덱스 리스트[-1] 뒤에서부터 세기

정의 음수를 쓰면 뒤에서부터 셉니다. 마지막이 -1, 그 앞이 -2.

```
temps = [25, 26, 24, 28, 27]
#       -5  -4  -3  -2  -1
print(temps[-1])   # 27 (마지막)
print(temps[-2])   # 28 (뒤에서 두 번째)
```

```
▶ 27
▶ 28
```

왜 쓰나 실전에서 -1을 압도적으로 많이 씁니다. 데이터는 대체로 뒤에 추가되므로 **마지막 값 = 가장 최신 값**이기 때문이에요. `len(temps) - 1`로도 되지만 번거로워서 -1을 씁니다.

응용 “최근 값 확인” 이 필요한 모니터링 코드에서 -1은 사실상 관용구입니다. 최근 몇 개를 보려면 다음 STEP의 슬라이싱 `[-3:]`으로 확장됩니다.

```
temps = [25, 26, 24, 28, 27]
latest = temps[-1]
print("최신 측정값:", latest)      # 27
print("직전 대비:", latest - temps[-2]) # -1 (하락)
```

▶ 최신 측정값: 27

▶ 직전 대비: -1

문법 인덱스로 꺼낸 값 계산하기 꺼내서 바로 연산

정의 인덱싱으로 꺼낸 값은 그냥 숫자이므로, 변수에 담아 사칙연산에 바로 쓸 수 있습니다.

```
output = [120, 95, 130, 110, 88, 102]
first = output[0]
last = output[-1]
print(first + last)      # 222
print((first + last) / 2) # 111.0 (중간값)
```

▶ 222

▶ 111.0

왜 쓰나 “첫 값과 마지막 값의 차이” 는 추세를 보는 가장 간단한 방법입니다. 실습 3이 바로 이 패턴이었어요.

설비 적용

설비 응용: `output[-1] - output[0]`으로 하루 동안의 변화량을, $(output[0] + output[-1]) / 2$ 로 대략적인 중간 수준을 봅니다. 값이 커지면 상승 추세, 음수면 하락 추세입니다.

STEP 3

슬라이싱 — 구간 잘라내기

콜론이 있으면 슬라이싱, 결과는 새 리스트

문법 슬라이싱 [start:end] 끝은 미포함

정의 콜론으로 범위를 지정해 여러 값을 잘라냅니다. 시작은 포함, 끝은 제외이며 결과는 **새 리스트**입니다.

```
temps = [25, 26, 24, 28, 27]
#       0   1   2   3   4
print(temps[1:4])    # [26, 24, 28]  (1,2,3번 / 4 제외)
print(temps[1:2])    # [26]   ← 값 하나여도 리스트!
```

```
▶ [26, 24, 28]
▶ [26]
```

왜 쓰나 인덱싱은 “값 하나”, 슬라이싱은 “리스트”를 돌려줍니다. `temps[1]`은 26이지만 `temps[1:2]`는 [26] — 겉보기 비슷해도 종류가 다릅니다.

개념

가져올 개수 = end - start. `temps[1:4]`는 4-1=3개입니다. 이 계산으로 미리 개수를 예측할 수 있어요.

문법 생략과 step [:n] [n:] [::k] 앞·뒤·건너뛰기

정의 시작을 비우면 처음부터, 끝을 비우면 끝까지, 콜론을 하나 더 붙이면 간격(step)을 지정합니다.

```
temps = [25, 26, 24, 28, 27, 23]
print(temps[:3])    # [25, 26, 24]  처음부터 3개
print(temps[2:])    # [24, 28, 27, 23]  3번째부터 끝까지
print(temps[-3:])   # [28, 27, 23]  뒤에서 3개
print(temps[::2])   # [25, 24, 27]  2칸씩 건너뛰기
```

```
▶ [25, 26, 24]
▶ [24, 28, 27, 23]
▶ [28, 27, 23]
▶ [25, 24, 27]
```

왜 쓰나 `[-3:]`(최근 3개)와 `[:3]`(앞 3개)이 실전에서 가장 많이 쓰입니다. step은 데이터가 너무 많을 때 일정 간격으로 “샘플링” 하는 데 유용해요.

응용 데이터를 앞·뒤 두 구간으로 나눠 비교하면 “전반부 대비 후반부” 추세를 볼 수 있습니다. 실습 5가 바로 이 패턴입니다.

```
hours = [3, 4, 5, 6, 5, 4, 6, 7, 8, 7, 5, 3]
first  = hours[:6]      # 앞 6개
second = hours[6:]      # 뒤 6개
print(sum(first)/6, sum(second)/6)  # 4.5 6.0 → 후반부 증가
print(temps[::-1])      # 리스트도 [::-1]로 뒤집기 가능
```

▶ 4.5 6.0

구분	인덱싱	슬라이싱
표기	temps[2]	temps[1:4]
콜론	없음	있음
결과	값 하나 24	리스트 [26, 24, 28]
범위 초과	IndexError	오류 없음

STEP 4

값 바꾸기와 존재 확인

리스트는 “바꿀 수 있는” 자료형이다

문법 인덱스로 값 바꾸기 그 자리만 교체

정의 등호 왼쪽에 인덱스를 두면 그 자리의 값만 교체됩니다. 문자열은 불가능하지만 리스트는 가능합니다(가변).

```
temps = [25, 26, 24, 28, 27]
temps[2] = 99
print(temps)    # [25, 26, 99, 28, 27]
▶ [25, 26, 99, 28, 27]
```

왜 쓰나 센서 오류로 들어온 이상값을 올바른 값으로 고칠 때 씁니다. 문자열은 `s[0]="X"`가 불가능(불변)한 것과 대조되는, 리스트의 핵심 성질이에요.

연산자 `in` · `not in` · `index()` 존재 확인 · 위치 찾기

정의 `in`은 값이 있는지 참·거짓으로 확인하고, `index()`는 그 값이 처음 나오는 위치를 알려줍니다.

```
machines = ["펌프", "모터", "압축기"]
print("모터" in machines)    # True
print("밸브" in machines)    # False
print("밸브" not in machines) # True
print(machines.index("모터")) # 1
▶ True
▶ False
▶ True
▶ 1
```

왜 쓰나 문자열에서 쓰던 `in`이 리스트에서도 그대로 통합니다. 다만 리스트에서는 “요소 하나와 통째로 같은지”를 봅니다.

응용 `in`으로 존재를 확인한 뒤 `index()`로 위치를 찾아 고치는 3단 콤보가 데이터 정리의 기본 패턴입니다. `index()`는 값이 없으면 `ValueError`를 내므로, 반드시 `in`으로 먼저 확인하세요.

```
temps = [25, 26, 240, 28, 27]    # 240은 이상값
if 240 in temps:                  # ① 있는지 확인
    i = temps.index(240)          # ② 위치 찾기
    temps[i] = 24                  # ③ 올바른 값으로 교체
print(temps)                      # [25, 26, 24, 28, 27]
```

▶ [25, 26, 24, 28, 27]

주의

`index()`는 “처음 나오는” 위치 하나만 줍니다. 같은 값이 여러 개면 나머지는 못 찾으니 주의하세요.

STEP 5

값 추가·제거

서랍에 넣고 빼기

메서드 `append()` · `insert()` · `extend()` 끝에 · 위치에 · 묶음으로

정의 `append(값)`은 맨 끝에 하나 추가, `insert(위치, 값)`은 지정 위치에 끼워 넣기(뒤는 밀림), `extend(리스트)`는 다른 리스트를 풀어서 이어붙입니다.

```
temps = []
temps.append(30)
print(temps)          # [30]
temps.insert(0, 28)   # 맨 앞에 끼워 넣기
print(temps)          # [28, 30]
temps.extend([31, 32]) # 묶음으로 이어붙이기
print(temps)          # [28, 30, 31, 32]
```

```
▶ [30]
▶ [28, 30]
▶ [28, 30, 31, 32]
```

왜 쓰나 셋 다 “추가”지만 위치와 단위가 다릅니다 — 끝에(`append`) / 원하는 자리(`insert`) / 여러 개 한꺼번에(`extend`).

응용 가장 많이 쓰는 건 압도적으로 `append`입니다. “빈 리스트 → 반복해서 `append`”가 데이터를 모으는 표준 패턴이에요. `extend`와 `append`의 차이도 꼭 구분하세요.

```
a = [1, 2]
a.append([3, 4])   # 리스트 자체를 한 덩어리로 넣음
print(a)           # [1, 2, [3, 4]] ← 중첩!
b = [1, 2]
b.extend([3, 4])   # 풀어서 이어붙임
print(b)           # [1, 2, 3, 4]
```

```
▶ [1, 2, [3, 4]]
▶ [1, 2, 3, 4]
```

자주 하는 실수

`extend()`는 값을 돌려주지 않습니다(`None`). `print(data.extend(new))`를 하면 `None`이 출력돼요 — 먼저 실행하고, 그 다음 줄에서 리스트를 출력해야 합니다.

메서드 `remove()` · `pop()` · `del` 값으로 · 위치로(반환) · 위치로

정의 `remove(값)`은 그 값과 같은 첫 번째 항목을 제거, `pop(위치)`는 그 자리 값을 꺼내 돌려주면서 제거, `del`은 인덱스로 삭제하되 값을 돌려주지 않습니다.

```
temps = [25, 26, 999, 24, 28]
temps.remove(999)      # 값 기준 제거
print(temps)           # [25, 26, 24, 28]
x = temps.pop(1)       # 위치 기준, 꺼낸 값 반환
print(x, temps)        # 26 [25, 24, 28]
del temps[0]           # 위치 기준 삭제 (점 없이!)
print(temps)           # [24, 28]
```

```
▶ [25, 26, 24, 28]
▶ 26 [25, 24, 28]
▶ [24, 28]
```

왜 쓰나 기준이 다릅니다 — 값을 알면 `remove`, 위치를 알면 `pop/del`. 꺼낸 값을 쓸 일이 있으면 `pop`, 그냥 지우면 `del`입니다.

응용 `del`은 메서드가 아니라 ****키워드****라 점(.)을 쓰지 않습니다. 그래서 슬라이싱과 결합해 “구간 삭제”가 가능한 게 강점이에요.

```
nums = [1, 2, 3, 4, 5, 6]
del nums[1:3]          # 구간 삭제
print(nums)            # [1, 4, 5, 6]
del nums[:]            # 전체 비우기
print(nums)            # []
```

```
▶ [1, 4, 5, 6]
▶ []
```

개념

`pop()`은 괄호를 비우면 마지막 값을 꺼냅니다(`pop()`). 이 성질로 리스트를 “쌓았다 꺼내는” 스택처럼 쓸 수 있어요.

STEP 6

정렬과 탐색

순서를 바꾸고, 세고, 찾기

메서드 `sort()` · `reverse()` 크기순 정렬 · 순서 뒤집기

정의 `sort()`는 작은 값부터(오름차순) 정렬하고 `reverse=True`를 주면 큰 값부터. `reverse()`는 크기와 무관하게 현재 순서를 그대로 뒤집습니다.

```
temps = [27, 24, 30, 22, 26]
temps.sort()
print(temps)          # [22, 24, 26, 27, 30]
temps.sort(reverse=True)
print(temps)          # [30, 27, 26, 24, 22]

order = [25, 30, 22, 28]
order.reverse()        # 순서만 뒤집기
print(order)          # [28, 22, 30, 25]
```

- ▶ [22, 24, 26, 27, 30]
- ▶ [30, 27, 26, 24, 22]
- ▶ [28, 22, 30, 25]

왜 쓰나 `sort`는 “크기순”, `reverse`는 “현재 순서 뒤집기”로 목적이 완전히 다릅니다. 정렬된 리스트를 `reverse`하면 내림차순이 되지만, 정렬 안 된 리스트를 `reverse`하면 그냥 거꾸로일 뿐이에요.

응용 `sort(reverse=True)` 후 슬라이싱하면 “상위 N개”를 뽑을 수 있습니다. 단, `sort()`는 ****원본을 바꿉니다****
— 원본을 지키려면 내장 함수 `sorted()`를 쓰세요.

```
temps = [27, 24, 30, 22, 26]
top3 = sorted(temps, reverse=True)[:3]  # 원본 보존
print(top3)          # [30, 27, 26] 상위 3개
print(temps)         # [27, 24, 30, 22, 26] 원본 그대로!
```

- ▶ [30, 27, 26]
- ▶ [27, 24, 30, 22, 26]

자주 하는 실수

문자열 메서드(`upper` 등)는 원본을 안 바꾸고 새 값을 돌려줬지만, ****리스트의 `sort`·`reverse`·`append`는 원본을 직접 바꾸고 `None`을 돌려줍니다.**** `temps = temps.sort()`라고 쓰면 `temps`가 `None`이 되어버려요.

메서드 count() · index() 개수 세기 · 위치 찾기

정의 count(값)은 그 값이 몇 번 나오는지, index(값)은 처음 나오는 위치를 알려줍니다.

```
results = ["정상", "이상", "정상", "정상", "이상"]
print(results.count("정상"))    # 3
print(results.index("이상"))    # 1
```

```
▶ 3
▶ 1
```

왜 쓰나 문자열에서 쓰던 count·find와 짝이 되는 기능입니다(리스트는 find 대신 index). 상태 라벨이 각각 몇 번 나왔는지 집계할 때 바로 쓰입니다.

응용 count()로 상태별 개수를 세면 간단한 “집계 리포트”가 됩니다. 비율까지 구하면 불량률이 나오죠.

```
results = ["정상", "이상", "정상", "정상", "이상"]
total = len(results)
bad = results.count("이상")
print(f"이상 {bad}건 / 전체 {total}건")
print(f"이상 비율 {bad/total*100:.1f}%")
```

```
▶ 이상 2건 / 전체 5건
▶ 이상 비율 40.0%
```


STEP 7

리스트 복사와 메모리 (심화)

오늘 수업에서 가장 강조된 함정

리스트를 다른 변수에 “복사” 했다고 생각했는데, 원본까지 같이 바뀌는 일이 생깁니다. 이유를 알면 다시는 안 틀립니다 — 변수에 담기는 건 “값” 이 아니라 “값이 사는 집 주소” 이기 때문입니다.

개념 = 로 넘기면 “같은 리스트” 를 가리킨다 참조(reference)

정의 `new = old`는 리스트를 복사하는 게 아니라, 같은 리스트를 가리키는 이름표를 하나 더 붙이는 것입니다. 그래서 한쪽을 바꾸면 양쪽 다 바뀝니다.

```
nums = [1, 2, 3]
new_nums = nums          # 복사가 아니라 "같은 집 주소"를 넘김
new_nums.append(999)
print(nums)               # [1, 2, 3, 999] ← 원본도 바뀜!
print(new_nums)           # [1, 2, 3, 999]
```

```
▶ [1, 2, 3, 999]
▶ [1, 2, 3, 999]
```

왜 쓰나 메모리에는 값마다 “주소” 가 있습니다. 변수는 그 주소를 적어둔 이름표예요. `new = old`는 이름표를 하나 더 만든 것일 뿐, 서랍장 자체는 여전히 하나입니다.

그림으로 보기 — 참조 vs 복사

① `new_nums = nums` → 이름표만 하나 더 (서랍장은 하나)

nums	→	[1, 2, 3, 999] 같은 리스트
new_nums	→	↑ 위와 같은 것을 가리킴

② `new2 = nums.copy()` → 서랍장을 통째로 새로 만들

nums	→	[1, 2, 3, 999] 원본
new2	→	[1, 2, 3, 999, 222] 독립된 사본

메서드 `copy()` 진짜 복사본 만들기

정의 `copy()`는 리스트를 통째로 새로 만들어 돌려줍니다. 이제 두 리스트는 서로 독립이라 한쪽을 바꿔도 다른 쪽은 그대로입니다.

```

nums = [1, 2, 3, 999]
new2 = nums.copy()      # 진짜 복사
new2.append(222)
print(nums)              # [1, 2, 3, 999]      원본 그대로
print(new2)              # [1, 2, 3, 999, 222]   사본만 변경

```

```

▶ [1, 2, 3, 999]
▶ [1, 2, 3, 999, 222]

```

왜 쓰나 원본 데이터를 보존한 채 “가공한 버전”을 따로 만들 때 필수입니다. 어제 배운 문자열은 불변이라 이런 문제가 없었지만, 리스트는 가변이라 반드시 신경 써야 해요.

응용 `copy()` 외에 슬라이싱 `nums[:]`이나 `list(nums)`로도 같은 효과를 냅니다. 셋 다 “새 리스트를 만든다”는 점에서 동일해요.

```

nums = [1, 2, 3]
a = nums.copy()      # 방법1 (가장 명확)
b = nums[:]          # 방법2 (슬라이싱은 새 리스트)
c = list(nums)       # 방법3
a.append(9)
print(nums, a)       # [1, 2, 3] [1, 2, 3, 9]

```

```

▶ [1, 2, 3] [1, 2, 3, 9]

```

주의

원본을 지켜야 하는 상황이면 “일단 `copy()`부터” 하는 습관이 안전합니다. 어제 pandas의 `df.copy()`를 배웠던 것과 정확히 같은 이유예요.

부록 A

리스트 메서드 총정리

용도별 한 장 정리

용도	메서드/문법	하는 일	원본 변경
만들기	[]	리스트 생성	—
크기	len()	값의 개수	—
꺼내기	lst[i] · lst[a:b]	값 하나 / 구간(새 리스트)	X
추가	append/insert/extend	끝 / 위치 / 묶음 잇기	O
제거	remove/pop/del	값 / 위치+반환 / 위치	O
정렬	sort/reverse	크기순 / 뒤집기	O
탐색	count/index/in	개수 / 위치 / 존재	X
복사	copy() · lst[:]	독립된 사본	X

개념

“원본 변경” 열이 핵심입니다. 추가·제거·정렬은 원본을 직접 바꾸고 None을 반환하므로 lst = lst.sort() 처럼 쓰면 안 됩니다. 반대로 슬라이싱·copy는 새 리스트를 돌려줘 원본이 안전해요.

문자열 vs 리스트 — 같은 것과 다른 것

기능	문자열	리스트
인덱싱·슬라이싱	동일하게 가능	동일하게 가능
in / count / len	가능	가능
위치 찾기	find() (없으면 -1)	index() (없으면 오류)
값 바꾸기	불가 (불변)	가능 (가변)
메서드 결과	새 값 반환(원본 유지)	원본을 직접 변경

부록 B

오류·함정 사전

오늘 코드에서 걸려 넘어지기 쉬운 곳

✗ 이렇게 하면	무슨 일	✓ 바로잡기
<code>temps[5]</code> (5개 리스트)	<code>IndexError</code>	<code>temps[4]</code> / <code>temps[-1]</code>
<code>new = old</code> 후 수정	원본까지 바뀜(참조)	<code>new = old.copy()</code>
<code>lst = lst.sort()</code>	<code>None</code> 이 됨	<code>lst.sort()</code> (그대로)
<code>print(a.extend(b))</code>	<code>None</code> 출력	실행 후 다음 줄에 출력
<code>a.append([3,4])</code>	중첩 <code>[1,2,[3,4]]</code>	<code>a.extend([3,4])</code>
<code>lst.index(없는값)</code>	<code>ValueError</code>	<code>if 값 in lst:</code> 먼저
<code>temps[1:2]</code> 는 값?	아니오 — <code>[26]</code> 리스트	값 하나는 <code>temps[1]</code>

주의

오늘의 1순위 디버깅 팁: “원본이 이상하게 바뀌었다” → 어딘가에서 `=` 로 리스트를 넘겨놓고 수정했는지 확인하세요. 리스트는 가변이라 이 함정이 자주 생깁니다.

부록 C

실습 하이라이트 & 다음 시간

내가 쓴 코드에서 한 걸음 더

실습에서 이미 강의 범위를 넘어선 문법을 몇 개 썼습니다. 지금은 “이런 게 있구나” 정도로 봐두면, 다음 단원에서 정식으로 배울 때 훨씬 쉽습니다.

① 리스트 컴프리헨션 — 한 줄로 리스트 만들기

```
import random
nums = [random.randint(1, 100) for _ in range(10)] # 1~100 정수 10개
# 뜻: range(10)만큼 반복하며 randint 결과를 리스트에 담아라
# _ 는 "쓰지 않는 변수"라는 관례적 이름
```

▶ [42, 7, 88, ...] (실행할 때마다 달라짐)

연결

다음 단원의 **반복문(for)**을 배우면 이 한 줄이 완전히 이해됩니다. 지금은 “반복해서 리스트를 채우는 축약 표현”으로 기억해 두세요.

② try / except — 오류가 나도 멈추지 않게

```
temps = [23, 26, 22, 25]
try:
    idx = temps.index(240) # 없으면 ValueError
    temps[idx] = 24
except ValueError:
    pass # 없으면 그냥 넘어감
print(temps)
```

▶ [23, 26, 22, 25]

개념

`if 240 in temps:`로 미리 확인하는 방식과 목적이 같습니다. 초보 단계에서는 **in**으로 먼저 확인하는 쪽이 읽기 쉬워 권장되고, try/except는 예외 처리를 배운 뒤 쓰면 좋습니다.

다음 시간 예고

다음 주제	한 줄 미리보기
조건문 (if / elif / else)	3일차에 만든 참·거짓으로 갈림길 결정 — “온도 90 초과면 경고”
반복문 (for)	리스트의 값을 하나씩 꺼내 처리 — 오늘의 컴프리헨션이 이해되는 순간
리스트 + 조건 + 반복	“이상값만 골라내기” 같은 진짜 데이터 필터링

연결

교안 04_01의 Part 2가 바로 조건문입니다. 오늘 만든 리스트에 **if(판단)**와 **for(반복)**가 붙으면, 드디어 “수천 개 데이터에서 이상값만 자동으로 찾아내는” 코드를 쓸 수 있게 됩니다.

맺음말

오늘의 성취와 다음 한 걸음

값 하나에서 값 목록으로

오늘로 “값 하나에 이름 하나”라는 제약을 벗어났습니다. 리스트에 값을 담고, 인덱스·슬라이싱으로 꺼내고, 넣고 빼고 정렬하고, 마지막으로 참조와 복사의 차이까지 이해했습니다. 특히 STEP 7의 `copy()` 개념은 앞으로 pandas로 데이터프레임을 다룰 때 그대로 다시 만나게 되는, 오래 쓰이는 지식입니다.

개념

복습 루틴 추천: ① ``09_list.py``를 위에서 아래로 다시 실행하며 각 메서드가 “원본을 바꾸는지”를 확인 → ② ``nums``와 ``nums.copy()``로 참조·복사 실험을 직접 재현 → ③ ``sorted(temps, reverse=True)[:3]``으로 상위 3개를 뽑아보기 → ④ ``results.count("이상")``으로 불량률까지 계산해보기. 각 메서드를 외우기보다 “원본을 바꾸나? 무엇을 돌려주나?” 두 질문으로 정리하면 오래 남습니다.

— 끝 —